01 - Fundamentos: Tipos, operadores y casting

Pregunta número 1

¿Cuál es la función principal del JDK (Java Development Kit)?

Seleccione una:

- a. Permitir la edición de archivos JHTML y JCSS.
- b. Servir como un servidor web para aplicaciones Java.
- c. Proporcionar un entorno de ejecución para aplicaciones Java.
- d. Ofrecer herramientas necesarias para compilar, depurar y ejecutar aplicaciones Java.
- e. Ninguna opción es correcta.

Respuesta: El JDK (Java Development Kit) es el kit de desarrollo completo para Java. Su función principal es proporcionar todo lo necesario para que un programador pueda crear aplicaciones Java. Podemos decir que es el paquete de software completo destinado a los programadores. Su propósito es proveer todo el conjunto de utilidades para la creación de software, lo que incluye el compilador (javac), herramientas de depuración (jdb), generadores de documentación (javadoc) y el propio motor de ejecución (java).

Pregunta número 2

¿Qué herramienta se utiliza para empaquetar clases de Java en un archivo JAR (Java Archive)?

Seleccione una:

- a. jconsole
- b. java
- c. jps
- d. javac
- e. jar

Respuesta: La herramienta jar (Java Archive) es la utilidad de línea de comandos incluida en el JDK (Java Development Kit) que se utiliza específicamente para empaquetar y comprimir múltiples archivos (como clases .class, imágenes, sonidos y metadatos) en un solo archivo con extensión .jar. Este formato está basado en el formato ZIP y se utiliza comúnmente para distribuir bibliotecas y aplicaciones Java.

Pregunta número 3

¿Cuál de las siguientes herramientas incluidas en el JDK se utiliza para monitorear y gestionar aplicaciones?

Seleccione una:

- a. VM
- b. javac
- c. jar
- d. jconsole
- e. java

Respuesta: jconsole es una herramienta gráfica de monitoreo y gestión que viene incluida en el JDK (Java Development Kit). Utiliza las extensiones de gestión de Java (JMX) para proporcionar información sobre el rendimiento y el consumo de recursos de una aplicación Java que se está ejecutando en una máquina virtual de Java (JVM). Permite monitorear el uso de memoria, la actividad de los hilos, la carga de clases y el uso de la CPU, entre otros.

Pregunta número 4

Which declaration initializes a boolean variable?

Seleccione una:

- a. boolean k = 0;
- b. boolean m = null;
- c. boolean h = 1;
- d. boolean j = (1 < 5);

Respuesta: En Java, el tipo de dato boolean (primitivo) solo puede contener dos valores posibles: true o false. A diferencia de otros lenguajes como C o C++, Java es un lenguaje fuertemente tipado y no permite usar números enteros (0 o 1) para representar valores booleanos. Vamos a analizar cada opción para saber cuál es la correcta y el porqué. El primer inciso es a. boolean k = 0; esta opción es incorrecta debido a que Java no permite asignar un número entero (int) directamente a una variable booleana. Esto causaría un error de compilación. El inciso b. boolean m = null también es incorrecto, ya que boolean es un tipo de dato primitivo y los tipos primitivos no pueden tener el valor null. Solo los objetos, como la clase envolvente Boolean, pueden ser null. El inciso c. boolean h = 1 también es incorrecto. Al igual que la opción a, no se puede asignar un int (1) a un boolean. Finalmente, llegamos

al inciso d. boolean j = (1 < 5), que es el único correcto. La expresión (1 < 5) es una operación de comparación relacional. El resultado de evaluar esta expresión es un valor lógico: como 1 es menor que 5, el resultado es true. Por lo tanto, se está asignando un valor booleano válido a la variable j.

Pregunta número 5

¿Qué declaraciones inicializan correctamente una variable booleana en Java?

Seleccione una o más de una:

- [ ] a. boolean d = (4 - 14);
- [ ] b. boolean f ="true";
- [ ] c. boolean b = (15 = 2);
- [x] d. boolean c = (7 >= 7);
- [x] e. boolean e = (10 > 5 && 2 < 1);
- [x] f. boolean a = (3 < 6);

Respuesta: En Java, una variable de tipo primitivo boolean solo puede almacenar dos valores posibles: true o false. No se puede asignar ningún otro tipo de dato (como números enteros o cadenas de texto) directamente a una variable booleana. Para inicializar correctamente una variable booleana, se debe usar una expresión que el compilador pueda evaluar y cuyo resultado sea un valor lógico (true o false). Esto se logra mediante operadores de comparación (<, >, <=, >=, ==, !=) u operadores lógicos (&&, ||, !). Hay que analizar cada opción para entender por qué son o no son incorrectas: Empezando con el inciso a. boolean d = (4 - 14), tenemos que esta opción es incorrecta. La expresión (4 - 14) es una operación aritmética que da como resultado el número entero -10. No se puede asignar un tipo int a un boolean, ya que nos dará error de compilación. El inciso b. boolean f = "true" también es incorrecto, ya que "true" está entre comillas dobles, por lo que es un objeto de tipo String (una cadena de texto), no un valor booleano. Esto nos dará error de compilación por incompatibilidad de tipos. El inciso c. boolean b = (15 = 2) es incorrecto, ya que en Java, el operador = se usa para asignación, no para comparación. Además, no se puede asignar un valor a un número literal. Para comparar igualdad se usa ==. Esta línea no compila. Ahora sí llegamos al primer inciso correcto, que es el d. boolean c = (7 >= 7). La expresión (7 >= 7) evalúa si 7 es mayor o igual que 7. Como esto es cierto, el resultado de la expresión es el valor booleano true, que se asigna correctamente a c. El siguiente inciso correcto es el e. boolean e = (10 > 5 && 2 < 1), ya que esta expresión utiliza operadores lógicos (&& significa "Y"). Se evalúa primero (10 > 5), que es true, y (2 < 1), que es false. Luego,

true && false da como resultado false, un valor booleano válido. Finalmente, el último inciso f. boolean a = (3 < 6) también es correcta. La expresión (3 < 6) evalúa si 3 es menor que 6. Como es cierto, el resultado es el valor booleano true, asignándose correctamente a `a`.

Pregunta número 6

¿Cuál es el tipo de resultado cuando se resta un valor short de un valor double en Java?

Seleccione una:

- a. short
- b. No se puede restar un short de un double.
- c. int
- d. float
- e. double

Respuesta: En este ejercicio se están evaluando las reglas de promoción numérica (numeric promotion) en las expresiones aritméticas de Java. Una regla de promoción nos dice que cuando se realiza una operación matemática binaria (como una resta) entre dos valores numéricos de tipos primitivos diferentes, el compilador de Java convierte automáticamente el tipo más pequeño o menos preciso al tipo de mayor capacidad antes de ejecutar la operación. En esta evaluación participan un valor de tipo short (un número entero de 16 bits) y un valor de tipo double (un número de punto flotante de 64 bits de doble precisión). Siguiendo la estricta jerarquía de promoción de tipos en Java, si cualquiera de los operandos en una expresión es un double, el otro operando (en este caso, el short) se promueve de forma implícita a double para evitar la pérdida de precisión. La resta se efectúa entonces entre dos valores double, por lo que el resultado final de la expresión hereda este mismo tipo.

Pregunta número 7

What is the result?

```
public class Main {
    public static void main(String[] args) {
        int a = 0;
        a++;
        System.out.println(a++);
        System.out.println(a);
    }
}
```

Seleccione una:

- a. 0 1
- b. 1 2
- c. 1 1
- d. 2 2

Respuesta: En este ejercicio se está evaluando el comportamiento del operador de post-incremento (++) en Java, específicamente la diferencia temporal entre la evaluación de una expresión y la actualización del valor en memoria. Hay que realizar el seguimiento del valor de la variable paso a paso. Primero se declara la variable de tipo entero a y se inicializa con el valor de 0. La instrucción a++ es una sentencia autónoma que incrementa el valor de la variable en 1. En este punto de la ejecución, `a` pasa a valer 1 en la memoria. Al llegar a la instrucción System.out.println(a++), entra en juego la regla principal del operador de post-incremento: el operador devuelve primero el valor actual de la variable para ser consumido por la función (la impresión) y después incrementa el valor subyacente. Por lo tanto, se imprime en consola el valor actual 1, e inmediatamente después el valor de a sube a 2 internamente. La última instrucción, System.out.println(a), se encarga simplemente de recuperar e imprimir el valor que la variable tiene almacenado en ese momento. Como fue incrementada en el paso anterior, imprime 2. El programa imprimirá secuencialmente los números 1 y 2.

Pregunta número 8

What is the result?

```
public class Test{
    public static void main(String[] args){
        int b = 4;
        b--;
        System.out.print(--b);
        System.out.println(b);
    }
}
```

Seleccione una:

- a. 32
- b. 12
- c. 22
- d. 33

Respuesta: En este ejercicio se evalúa la diferencia fundamental entre los operadores de pre-decremento (--b) y post-decremento (b--) en Java, así como el comportamiento de la impresión estándar sin salto de línea. Vamos a trazar el valor de la variable en memoria paso a paso para entender lo que está sucediendo. Primero se declara la variable local b de tipo entero y se inicializa con el valor de 4. De ahí nos vamos al decremento independiente: La instrucción b--; reduce el valor de la variable en 1. En este punto, b pasa a valer 3 en la memoria. Al estar en una línea autónoma sin ser evaluada por otra función, el efecto inmediato es simplemente actualizar el valor almacenado. Continuamos con el pre-decremento e impresión: La instrucción System.out.print(--b); utiliza el operador de pre-decremento. La regla del lenguaje dicta que primero se reduce el valor de la variable en memoria (pasando de 3 a 2) y después se devuelve ese nuevo valor para ser consumido por el método de impresión. Es importante notar el uso de print() en lugar de println(), lo que significa que el cursor de la consola se queda en la misma línea tras imprimir el 2. La última instrucción System.out.println(b); recupera el valor actual de b, que es 2, y lo imprime adyacente a la salida anterior.

Pregunta número 9

¿Qué imprime este código?

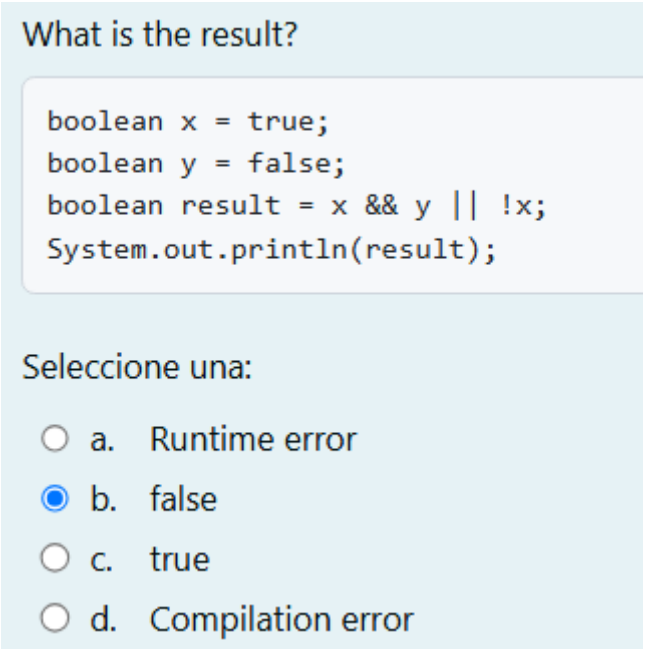
```
public static void main(String[] args) {
    boolean a = true;
    boolean b = false;
    boolean c = a && b || a;
    System.out.println(c);
}
```

Seleccione una:

- a. false
- b. Excepción
- c. true
- d. null

Respuesta: En este código se evalúan las reglas de precedencia de los operadores lógicos en Java. Vamos a desglosar la evaluación de la expresión paso a paso para entender el código. Primero tenemos que las variables locales se inicializan explícitamente con los valores a = true y b = false. En la instrucción boolean c = a && b || a, intervienen el operador lógico de cortocircuito AND (&&) y el operador lógico OR (||). En la jerarquía de operadores de Java, el operador && tiene una precedencia mayor que el operador ||. Debido a esta precedencia, la evaluación se agrupa implícitamente como (a && b) || a. El programa resuelve primero (true && false), lo cual da como resultado temporal false. La expresión general se reduce entonces a false || a, lo que equivale a false || true. Dado que el operador OR devuelve verdadero si al menos uno de los dos operandos es verdadero, el resultado final de toda la evaluación es true. Entonces, el valor true se asigna a la variable c y la instrucción System.out.println(c); lo muestra en la consola.

Pregunta número 10



Respuesta: En este ejercicio se evalúan las reglas de precedencia de los operadores lógicos en Java, incorporando el operador de negación unaria (!). Hay que desglosar la evaluación de la expresión booleana `result = x && y || !x` paso a paso, respetando la jerarquía de operadores. Las variables se declaran e inicializan con los valores x = true e y = false. En Java, el operador lógico unario de negación (!) tiene la mayor precedencia entre los operadores lógicos. El programa evalúa primero !x (que equivale a !true), convirtiéndose en false. En este punto, la expresión conceptualmente se reduce a x && y || false. El siguiente en la jerarquía de precedencia es el operador AND (&&). Se evalúa la parte izquierda de la expresión: x && y (que equivale a true && false). El resultado de esta operación es false. La expresión completa ahora se reduce a false || false. Finalmente, al tener la menor precedencia, se evalúa el operador OR. La operación final es false || false, cuyo resultado lógico es estrictamente false. Finalmente, el valor evaluado (false) se asigna a la variable result y se imprime directamente en la consola mediante System.out.println.

Pregunta número 11

Nivel: **Básico**

Whats is the result?

```
public static void main(String[] args) {
    System.out.println("Result:" + 3 + 5);
    System.out.println("Result:" + (3 + 5));
}
```

Seleccione una:

- a. Result: 8 Result: 8
- b. Result: 35 Result: 8
- c. Result: 8 Result: 35
- d. Result: 35 Result: 35

Respuesta: En este ejercicio se evalúan las reglas de precedencia, la asociatividad de izquierda a derecha y el comportamiento de la sobrecarga del operador + cuando interactúan cadenas de texto (String) y tipos numéricos. Empecemos con la primera instrucción ("Result:" + 3 + 5): Cuando operadores de la misma precedencia (como múltiples +) están en secuencia, Java evalúa la expresión de izquierda a derecha. Primero evalúa "Result:" + 3. Al detectar que el lado izquierdo es un String, el compilador trata el operador + como una concatenación de cadenas, convirtiendo el 3 a texto para producir "Result:3". A continuación, evalúa "Result:3" + 5. Siguiendo la misma regla, concatena el 5 al final de la cadena existente, resultando en la impresión de Result:35. Vamos con la segunda instrucción ("Result:" + (3 + 5)): El uso de los paréntesis altera el orden natural de evaluación, otorgando a su contenido la mayor prioridad. El programa resuelve primero (3 + 5). Al ser ambos operandos numéricos, el operador + actúa como suma matemática tradicional, dando como resultado 8. Luego, el compilador evalúa la operación exterior: "Result:" + 8. Al igual que antes, detecta una cadena e interactúa mediante concatenación, imprimiendo finalmente Result:8.

Pregunta número 12

What is the result?

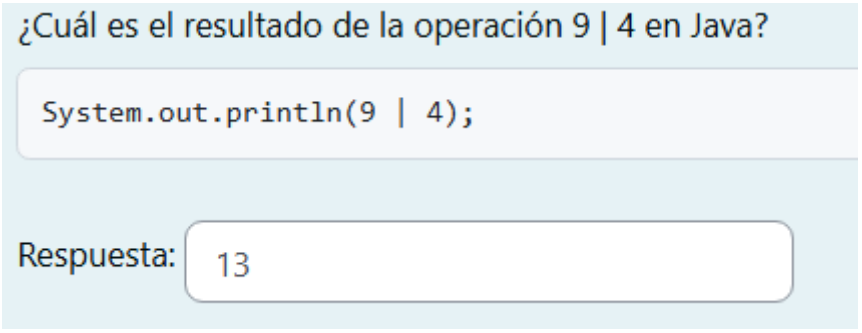
```
public static void main(String[] args) {
    System.out.println("Result: " + 2 + 3 + 5);
    System.out.println("Result: " + 2 + 3 * 5);
}
```

Seleccione una:

- a. Result: 10 Result: 30
- b. Result: 25 Result: 10
- c. Result: 235 Result: 215
- d. Result: 215 Result: 215
- e. Compilation fails.

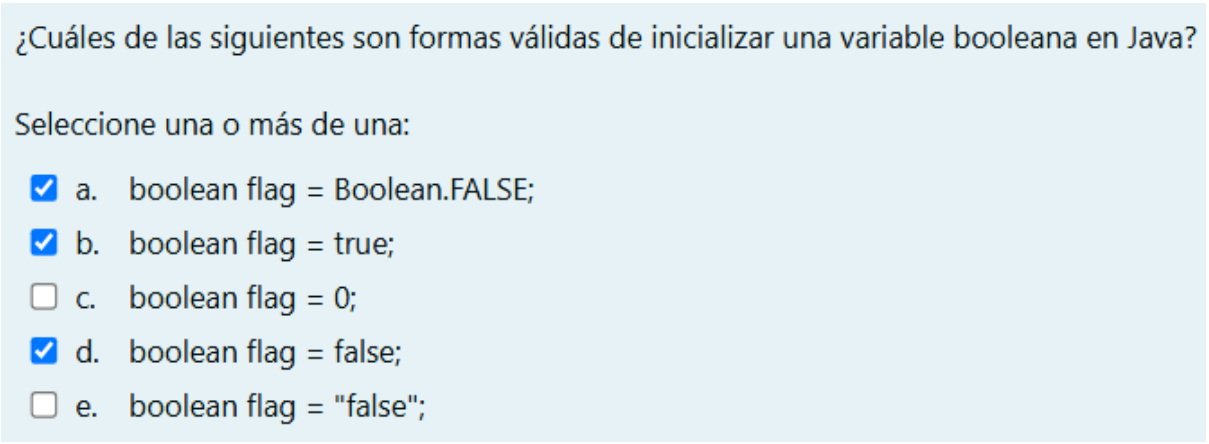
Respuesta: En este ejercicio se evalúan las reglas de precedencia de operadores y la evaluación de expresiones de izquierda a derecha cuando se combinan cadenas de texto y operaciones aritméticas en Java. Vamos con la primera expresión ("Result: " + 2 + 3 + 5): Al contener únicamente operadores de suma (+), todos comparten el mismo nivel de precedencia. Por lo tanto, la expresión se evalúa estrictamente de izquierda a derecha. Primero, se evalúa "Result: " + 2. Como el operando izquierdo es una cadena de texto (String), el operador actúa como concatenación, produciendo la cadena temporal "Result: 2". Luego, "Result: 2" + 3 vuelve a aplicar la concatenación debido a la presencia de la cadena, generando "Result: 23". Finalmente, "Result: 23" + 5 concatena el último dígito, resultando en la impresión de "Result: 235". Continuemos con la segunda expresión ("Result: " + 2 + 3 * 5): Aquí intervienen los operadores + y *. En la jerarquía de operadores de Java, la multiplicación tiene una precedencia estrictamente mayor que la suma o la concatenación. El programa resuelve primero la operación matemática aislada 3 * 5, cuyo resultado es el número entero 15. La expresión completa se reduce lógicamente a "Result: " + 2 + 15. Retomando la evaluación estándar de izquierda a derecha, se ejecuta la primera operación: "Result: " + 2, resultando en la cadena "Result: 2". Por último, se concatena el número previamente calculado: "Result: 2" + 15, lo que genera la salida "Result: 215".

Pregunta número 13



Respuesta: En este ejercicio se evalúa el conocimiento sobre los operadores lógicos a nivel de bits (bitwise) en Java, específicamente el operador OR inclusivo (|). A diferencia del operador lógico de cortocircuito (||), que evalúa condiciones booleanas, el operador simple (|) trabaja directamente con la representación binaria de los números enteros. Vamos a resolver la operación paso a paso. Empecemos con la conversión a binario: El programa primero convierte ambos operandos decimales a su formato binario. El número decimal 9 se representa como 1001. El número decimal 4 se representa como 0100. Ahora vamos con la operación OR bit a bit: El operador compara cada columna de bits de derecha a izquierda. La regla del OR bit a bit dicta que si al menos uno de los bits en la columna es 1, el bit resultante será 1; si ambos son 0, el resultado es 0. Entonces tenemos que 9 en binario es 1001 y que 4 en binario es 0100. Finalizamos con la conversión a decimal: El resultado de la operación a nivel de bits es la secuencia binaria 1101. Al convertir este valor de vuelta al sistema decimal tradicional, calculamos los valores posicionales: 8 + 4 + 0 + 1 = 13.

Pregunta número 14



Respuesta: En Java, una variable de tipo primitivo boolean solo puede almacenar los valores true o false. Además, Java es un lenguaje de tipado fuerte, lo que significa que no permite conversiones automáticas de otros tipos de datos (como

números enteros o cadenas de texto) a booleanos. Vayamos analizando cada opción para encontrar las respuestas correctas. Primero tenemos el inciso a. boolean flag = Boolean.FALSE; esta opción es correcta. Debemos recordar que Boolean es la clase envolvente (wrapper) del tipo primitivo boolean. Boolean.FALSE es una constante que representa el valor lógico falso. Es importante recordar que existe el autoboxing/unboxing, lo que permite asignar directamente un objeto Boolean a una variable primitiva boolean. Java automáticamente "desempaqueta" el objeto para extraer su valor primitivo false. El inciso b. boolean flag = true; también es correcto. Es la forma literal estándar de inicializar una variable booleana con el valor verdadero. Continuando, llegamos al inciso c. boolean flag = 0, que es la primera opción incorrecta. A diferencia de lenguajes como C o C++ (donde 0 es false y 1 es true), Java no permite asignar números enteros a variables booleanas. Intentar hacer esto provocará un error de compilación. El siguiente inciso es d. boolean flag = false; el cual es correcto, ya que es la forma literal estándar de inicializar una variable booleana con el valor falso. El siguiente inciso es el e. boolean flag = "false"; el cual es incorrecto. Esto se debe a que "false" está entre comillas dobles, por lo que es un objeto de tipo String (cadena de texto), no un valor booleano. Java no convierte cadenas de texto a booleanos automáticamente. Si se quisiera usar ese texto, se tendría que parsear explícitamente: boolean flag = Boolean.parseBoolean("false"). Tal como está escrita, la línea da un error de compilación.

Pregunta número 15

¿Cuáles de los siguientes operadores son válidos para variables booleanas en Java?

Seleccione una o más de una:

- [x] a. &&
- [x] b. |
- [ ] c. +
- [x] d. ||
- [x] e. ==

Respuesta: En este ejercicio se evalúa la compatibilidad de los distintos operadores de Java con el tipo de dato primitivo boolean. Empezamos con el inciso a. && y d. ||: Son los operadores lógicos condicionales (de cortocircuito) por excelencia. Están diseñados específicamente para conectar y evaluar expresiones booleanas. Después tenemos el inciso b. |: Aunque es comúnmente utilizado como operador OR a nivel de bits para números enteros, en Java también es un operador lógico completamente válido para valores booleanos (al igual que su contraparte &). La diferencia fundamental con || es que este operador lógico simple no realiza cortocircuito; siempre evaluará ambos lados de la expresión obligatoriamente.

Finalmente, el inciso e. ==: El operador relacional de igualdad se puede utilizar sin problemas para comparar si dos variables booleanas poseen el mismo estado (true o false). La única opción incorrecta es el inciso c. + ya que en Java, el operador de adición matemática o concatenación de cadenas no está definido para operandos de tipo boolean. A diferencia de lenguajes como C++, no se pueden sumar valores de verdad ni convertirlos implícitamente a valores numéricos.

Pregunta número 16

Which two declarations will compile?

```
14.    public static void main(String[] args) {
15.        Int a, b, c = 0;
16.        int a, b, c;
17.        int g, int h, int i = 0;
18.        int d, e, f;
19.        Int k, l, m, = 0;
20.    }
```

Seleccione una o más de una:

- [ ] a. Line 15.
- [x] b. Line 16.
- [ ] c. Line 17.
- [x] d. Line 18.
- [ ] e. Line 19.
- [ ] f. Line 20.

Respuesta: En Java, la sintaxis para declarar variables de un mismo tipo en una sola línea requiere escribir el tipo de dato una sola vez, seguido de una lista de nombres de variables separados por comas. Además, Java es un lenguaje case-sensitive (distingue entre mayúsculas y minúsculas). Hay que analizar cada línea para entender bien lo que está pasando y saber cuáles son los incisos correctos. Primeramente, tenemos el inciso a. Línea 15 (Int a, b, c = 0;). La palabra clave para el tipo de dato entero es int (todo en minúsculas). Int (con mayúscula) no es un tipo primitivo válido en Java y provocará un error de compilación; por lo tanto, este inciso es incorrecto. Seguimos con el inciso b. Línea 16 (int a, b, c;). Esta es una declaración válida. Se declara el tipo int una vez y luego los tres nombres de variables separados por comas, por lo que esta es una opción correcta. Vamos con el inciso c. Línea 17 (int g, int h, int i = 0;). Recordemos que no se debe repetir el tipo de dato (int) para cada variable en la misma declaración. La sintaxis correcta

sería int g, h, i = 0. Esto provocaría un error de compilación, por lo que esta opción es incorrecta. El siguiente inciso es el d. Línea 18 (int d, e, f;). Al igual que la línea 16, es una declaración múltiple perfectamente válida; con eso sabemos que esta opción es correcta. El siguiente inciso en la lista es el e. Line 19 (Int k, l, m, = 0;): aquí tenemos dos errores. Primero, usa Int con mayúscula y, segundo, hay una coma extra antes del signo igual (m, = 0), lo cual es un error de sintaxis; con esto dicho, sabemos que esta opción es incorrecta. Finalmente, vayamos al inciso f. Línea 20 (}): Aquí no hay mucho que decir, solo tenemos una llave de cierre, no una declaración de variable; así que es una opción incorrecta.

Pregunta número 17

Cuando se asigna una variable byte (no final) a un carácter (char) en Java sin cast explícito, ¿cuál es el tipo de resultado?

Seleccione una:

- a. char
- b. int
- c. byte
- d. short
- e. No puedes asignar un byte a un carácter.

Respuesta: En Java, la conversión automática (implícita) entre tipos primitivos solo se permite cuando se trata de una conversión de ampliación (widening conversion), es decir, cuando se pasa de un tipo de dato de menor capacidad a uno de mayor capacidad, como de byte a short, int, long, float o double. Hay que ver el caso de asignar un byte a un char: Tenemos que byte es un entero de 8 bits con signo (rango de -128 a 127), mientras que char es un entero de 16 bits sin signo (rango de 0 a 65535). Aunque un char tiene mayor capacidad (16 bits) que un byte (8 bits), la conversión no está permitida automáticamente porque no se considera una ampliación de un subtipo a un supertipo (el byte no es un subtipo del char). Además, representan rangos diferentes: un byte puede contener números negativos que un char no puede representar. Para realizarla, se requiere un cast explícito (ej. char c = (char) miByte;), ya que podría haber pérdida de información para valores negativos.
Algo a resaltar es que Java permite la asignación directa de un byte a un char solo si la variable byte es declarada como final y su valor es una constante conocida en tiempo de compilación que esté dentro del rango positivo válido del char (0 a 127). Como la pregunta especifica explícitamente "(no final)", esta excepción no aplica, y la asignación fallará en tiempo de compilación.

Pregunta número 18

What is printed out when the program is executed?

```
public class MainMethod{
    void main(){
        System.out.println("one");
    }
    static void main(String args){
        System.out.println("two");
    }
    public static final void main(String[] args){
        System.out.println("three");
    }
    void mina(Object[] args){
        System.out.println("four");
    }
}
```

Seleccione una:

- a. There is no output
- b. three
- c. two
- d. four
- e. one

Respuesta: Para que una clase Java pueda ser ejecutada por la Máquina Virtual de Java (JVM), debe contener un método main que cumpla con una firma específica y estricta, la cual es: public static void main(String[] args). Analicemos cada uno de los métodos main declarados en la clase MainMethod para ver cuál es el correcto. Empezamos con void main(): Este no es válido. No es static ni public, y no recibe un arreglo de strings. La JVM no lo reconocerá como punto de entrada. Seguimos con static void main(String args), el cual tampoco es válido. El parámetro debe ser un arreglo de strings (String[]). Aquí se declaró como un string simple. Aunque el nombre y el tipo base son correctos, la falta de corchetes [] hace que la firma no coincida. Llegamos a public static final void main(String[] args). Este es completamente válido. Cumple con todos los requisitos: es public, es static, devuelve void y recibe un arreglo de Strings (String[]). La palabra clave final (que

indica que el método no puede ser sobrescrito) no impide que sea el punto de entrada. La JVM lo ejecutará. El siguiente en la lista es void mina(Object[] args): Este no es válido. El nombre del método es mina, no main. Además, no es static ni public, y el parámetro es un arreglo de Object. En conclusión, al ejecutar la clase, la JVM buscará el método con la firma correcta y encontrará únicamente el tercer método. Por lo tanto, se ejecutará ese método y se imprimirá three en la consola.

Pregunta número 19

Which two will compile, and can be run successfully using the following command?
java Fred1 hello walls

Seleccione una o más de una:

- [ ] a. abstract class Fred1 { public static void main(String[] args) { System.out.println(args[2]); } }
- [ ] b. class Fred1 { public static void main(String args) { System.out.println(args[1]); } }
- [x] c. class Fred1 { public static void main(String[] args) { System.out.println(args); } }
- [x] d. class Fred1 { public static void main(String[] args) { System.out.println(args[1]); } }

Respuesta: En este ejercicio se evalúa la firma obligatoria del método main, el paso de argumentos por línea de comandos y el conocimiento sobre los límites de los arreglos para evitar excepciones en tiempo de ejecución. Al lanzar el comando java Fred1 hello walls, la JVM inicializa el arreglo args con exactamente dos elementos: args[0] corresponde a "hello" y args[1] corresponde a "walls". Analizando cada opción, entenderemos mejor qué está pasando. Iniciemos con el inciso a: Aunque una clase declarada como abstract puede contener un método main estático válido y compilará sin problemas, no podrá ejecutarse exitosamente. Intenta imprimir args[2], un índice fuera de los límites del arreglo, lo que lanzará un ArrayIndexOutOfBoundsException en tiempo de ejecución. Seguimos con el inciso b: Este código compila, pero la firma del método es incorrecta. La JVM exige estrictamente que el punto de entrada reciba un arreglo (String[] args o varargs String... args). Al definir solo un parámetro String, la JVM no lo reconocerá como el punto de inicio y arrojará un error indicando que el método principal no fue encontrado. Llegamos al inciso c, el cual es nuestra opción correcta: La sintaxis y la firma son perfectas. Al pasar la variable args directamente a println, el programa no arroja ninguna excepción; simplemente se ejecuta con éxito e imprime la referencia del objeto en memoria (una cadena similar a [Ljava.lang.String;@...). La otra opción correcta es el inciso d, ya que la firma es válida y accede de manera segura a args[1]. El programa compila y se ejecuta exitosamente, mostrando la palabra "walls" en la consola.

Pregunta número 20

What is the narrowest valid return type for the calculate Method?

```
public class Test {
    ___ doMath (long l, short s) {
        return s / (l * 10);
    }
}
```

Seleccione una:

- a. float
- b. short
- c. int
- d. long
- e. double
- f. byte

Respuesta: En este ejercicio se evalúan las reglas de promoción numérica (numeric promotion) de Java al evaluar expresiones aritméticas con tipos primitivos de distinto tamaño. Vayamos a desglosar la expresión return s / (l * 10); paso a paso para entender qué está pasando. Empecemos con la evaluación de los paréntesis (l * 10): La variable l es de tipo long (64 bits) y el literal numérico 10 es de tipo int por defecto (32 bits). Cuando se realiza una operación binaria entre distintos tipos enteros, Java promueve automáticamente el tipo más pequeño al tipo más grande. Por lo tanto, el int se promueve a long, y el resultado temporal de esta multiplicación es un long. Sigamos con la evaluación de la división s / (...): Ahora el compilador se enfrenta a la división entre la variable s (de tipo short, 16 bits) y el resultado anterior (de tipo long, 64 bits). Siguiendo la misma regla de promoción, el short se convierte implícitamente en un long antes de realizar la división. Dado que la operación matemática completa da como resultado un valor de tipo long, el método debe declarar su tipo de retorno como long para que el código compile sin arrojar un error de posible pérdida de precisión. Si intentáramos devolver un int o short sin aplicar un cast explícito (int) o (short), la compilación fallaría.

Pregunta número 21

What is the narrowest valid return type for the doMath method?

```
public class Test {
    ___ doMath (long l, short s) {
        return s / (l * 10);
    }
}
```

Seleccione una:

- a. float
- b. short
- c. int
- d. long
- e. double
- f. byte

Respuesta: Este ejercicio evalúa exactamente el mismo concepto que el anterior y, de hecho, es el mismo código que el que se nos presenta en el ejercicio anterior; lo único que cambia es la pregunta, entonces solo tenemos que recordar las reglas de promoción numérica (numeric promotion) de Java en expresiones aritméticas con tipos primitivos de distinto tamaño. Vamos a desglosar rápidamente la expresión return s / (l * 10); paso a paso. Comenzamos con la evaluación de los paréntesis (l * 10): La variable l es de tipo long (64 bits) y el literal numérico 10 es de tipo int por defecto (32 bits). En Java, cuando se opera con distintos tipos enteros, el más pequeño se promueve al más grande. El int se promueve a long, haciendo que el resultado de esta multiplicación sea un long. Seguimos con la evaluación de la división s / (...): Se divide la variable s (de tipo short, 16 bits) entre el resultado anterior (de tipo long, 64 bits). De nuevo, el short es promovido a long antes de ejecutar la división. Puesto que la operación completa produce un valor long, el tipo de retorno más estricto (narrowest) que puede declararse en la firma del método sin requerir un cast explícito es long. Si se intentara retornar un int o short, el compilador arrojaría un error por posible pérdida de precisión.

Pregunta número 22

Dado el siguiente código ¿Cuál es el tipo de retorno válido punto flotante más restringido para el método de cálculo?

```
public class Test {
    _____ doMath(long l, short s) {
        return s / (l * 10);
    }
}
```

Seleccione una:

- a. long
- b. float
- c. short
- d. int
- e. byte
- f. double

Respuesta: Para determinar el tipo de retorno válido más restringido, debemos analizar dos cosas: el tipo de dato que resulta de la expresión matemática y las reglas de conversión de tipos en Java. Vamos primero con el tipo de dato de la expresión: Analicemos la línea return s / (l * 10); aplicando las reglas de promoción numérica de Java: s es de tipo short, l es de tipo long y 10 es un literal de tipo int.
Paso A: Se evalúa l * 10. Como l es long y 10 es int, el int se promueve a long. El resultado de la multiplicación es de tipo long. Paso B: Se evalúa s / (resultado long). Como s es short y el otro operando es long, el short se promueve a long. El resultado final de toda la expresión es de tipo long. Ahora vamos con los tipos de retorno válidos (conversión de ampliación): El método debe devolver un valor de tipo long. En Java, un long se puede convertir automáticamente (sin necesidad de un cast explícito) a los siguientes tipos: long, float y double. Ahora vamos a filtrar por la restricción "punto flotante más restringido": La pregunta especifica explícitamente que busquemos un tipo de retorno de punto flotante. De los tipos válidos (long, float, double), los que son de punto flotante son float y double. Sabemos que float tiene 32 bits de precisión y double tiene 64 bits de precisión. El tipo de punto flotante "más restringido" (el de menor tamaño y menor rango) es float.

Pregunta número 23

Which of the following lines of code correctly defines the main method in a Java program?

Seleccione una o más de una:

- [ ] a. static void main(String args[]) {}
- [ ] b. public static int main(String[] args) {}
- [x] c. public static void main(String... String) {}
- [x] d. public static void main(String[] args) {}
- [ ] e. private static void main(String[] args) {}

Respuesta: Para que la Máquina Virtual de Java (JVM) reconozca un método como el punto de entrada de un programa, este debe cumplir con una firma específica, la cual es public static void main(String[] args). Hay que analizar cada opción para ver cuáles son correctas y cuáles no. Empecemos con el inciso a. static void main(String args[]) {}. Esta es una opción incorrecta, ya que falta el modificador de acceso public. Aunque el código compila, la JVM no lo reconocerá como el método main para iniciar la ejecución, por lo que dará un error en tiempo de ejecución indicando que no se encuentra el método main. Seguimos con el inciso b. public static int main(String[] args) {}. Esta opción también es incorrecta, ya que el tipo de retorno es int, pero el método main debe devolver obligatoriamente void. Esto provoca un error de compilación. Continuamos con el inciso c. public static void main(String... String) {} el cual sí es correcto. La sintaxis String... (varargs) es equivalente a String[] y puede usarse en la firma del método main. Además, nombrar al parámetro String es técnicamente legal en Java, aunque es una mala práctica porque oculta el nombre de la clase String. Aun así, el código compila y la JVM lo ejecuta correctamente. El otro inciso correcto es el d. public static void main(String[] args) {}, ya que es la firma estándar y canónica del método main. Cumple con todos los requisitos: es public, static, devuelve void y recibe un arreglo de Strings (String[]). Finalmente, el inciso e. private static void main(String[] args) {} es incorrecto, ya que el modificador private restringe el acceso al método. La JVM necesita llamar a este método desde fuera de la clase, por lo que debe ser obligatoriamente public. Aunque compila, fallará en tiempo de ejecución al no encontrar el método main accesible.

Pregunta número 24

Which of the following statements about the java -jar command, which is used to execute Java programs packaged in JAR (Java Archive) files, are true?

Seleccione una o más de una:

- [ ] a. The java -jar command can dynamically add libraries to the classpath using the -cp or -classpath options.
- [x] b. The java -jar command ignores the CLASSPATH environment variable and command-line classpath settings.
- [ ] c. The java -jar command can execute a JAR file without specifying the main class if the JAR file contains multiple main classes.
- [x] d. The java -jar command requires the JAR file to have a META-INF/MANIFEST.MF file specifying the main class.
- [x] e. The java -jar command can only be used to execute JAR files that are executable JARs, i.e., those that contain the META-INF/MANIFEST.MF file with the Main-Class attribute.

Respuesta: En este ejercicio se evalúa el comportamiento de la JVM al ejecutar aplicaciones empaquetadas utilizando el flag -jar. Cuando utilizas este comando, delegas la configuración del entorno de ejecución de forma exclusiva al propio archivo JAR. Empecemos remarcando que las opciones d y e son correctas y se complementan. Para que un archivo JAR sea ejecutable directamente con java -jar, es un requisito estricto que contenga un archivo de metadatos ubicado en META-INF/MANIFEST.MF. Este manifiesto debe incluir obligatoriamente el atributo Main-Class apuntando a la clase que contiene el punto de entrada principal. Si no existe, la JVM lanzará un error de tipo "no main manifest attribute". La opción b también es correcta. Una regla fundamental es que, al usar el flag -jar, la JVM ignora por completo la variable de entorno estática CLASSPATH y cualquier parámetro -cp o -classpath que intentes pasar en la consola. La JVM asume que las dependencias adicionales están declaradas internamente a través del atributo Class-Path dentro del manifiesto. La opción a queda descartada por la misma regla explicada en el punto anterior; la adición dinámica con -cp es anulada por el flag -jar. La opción c también es incorrecta. Aunque un JAR esté compuesto por múltiples clases que contengan métodos main(), el comando java -jar no tiene capacidad de resolución ambigua ni ejecuta "sin especificar". Se regirá estrictamente por la única clase definida en el manifiesto. Si se desea ejecutar otra clase principal del mismo JAR, se debe evitar el uso de -jar y ejecutar mediante -cp.